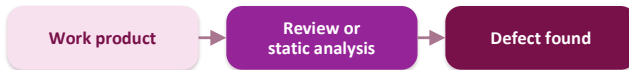


3. Static Testing

A. Static vs dynamic testing

Static testing



Dynamic testing



- Static testing finds **defects directly**; dynamic testing causes **failures**, which are then traced back to defects
- Static testing can reach code paths that rarely run, and work products that can't be run at all
- Static testing can measure qualities such as **maintainability**; dynamic testing measures qualities such as **performance**

Defects that are easier to find statically: unclear, missing or contradictory requirements; design flaws; coding defects such as undefined variables and unreachable code; deviations from standards; interface mismatches; security vulnerabilities; gaps in test basis coverage.

B. Review process (ISO/IEC 20246)



1. **Planning:** agree the scope, purpose, quality characteristics to check, exit criteria, effort and timeframe
2. **Review initiation:** make sure everyone has access to the work product, understands their role and is ready
3. **Individual review:** each reviewer checks the work product and logs anomalies, recommendations and questions (e.g., using a checklist or scenarios)
4. **Communication and analysis:** discuss each **anomaly** and decide its status, owner and action (not every anomaly is a defect)
5. **Fixing and reporting:** raise defect reports, fix defects, and accept the work product once the exit criteria are met

C. Static testing basics

In **static testing** the software is **not run**. Work products are examined by people (**reviews**) or by tools (**static analysis**). It supports both verification and validation.

- **Almost any work product can be examined:** requirements, backlog items, code, models, test plans, test cases and contracts
- **Reviews** need something people can read and understand; **static analysis** needs a formal structure, such as code or models
- **Not suitable:** work products that are hard to interpret, such as third-party executable code
- Static analysis often runs as part of **CI**, and can also check maintainability and security

D. Early and frequent stakeholder feedback

Getting feedback from stakeholders **early and often** stops the product drifting away from what they want, which would cause rework, missed deadlines and blame.

- Prevents misunderstandings about requirements
- Changes to requirements are understood and made earlier
- The team better understands what it's building
- Effort goes into the most valuable features

E. Review roles



Each role has a clear job, and one person can hold several roles. Don't confuse the **review leader**, who organises the review, with the **moderator**, who runs the review meeting.

F. Value of static testing

- Finds defects **early**, when they're cheapest to fix
- Finds defects dynamic testing can't, e.g., unreachable code, design patterns not followed, defects in documents
- Builds a shared understanding of work products, and confidence in them
- Lets stakeholders check that requirements describe their **real needs**
- Reviews cost money, but usually **reduce overall project costs** because less rework is needed

G. Review types

- **Informal review:** no defined process or formal output; the aim is to find anomalies
- **Walkthrough:** led by the author; used to educate reviewers, build consensus, generate ideas and find anomalies; preparation is optional
- **Technical review:** led by a moderator, with technically qualified reviewers; aims to reach consensus and make decisions on a technical problem
- **Inspection:** the most formal type; the author can't lead it; aims to find as many anomalies as possible, and metrics are collected

H. Success factors for reviews

- Clear objectives and measurable exit criteria (never use reviews to judge people)
- Choose the right review type
- Review in small chunks
- Give feedback to authors and stakeholders
- Allow time to prepare
- Get management support
- Make reviews part of the culture
- Train participants
- Facilitate meetings well